AMYPO - Policy & Placement Copilot

Product & Engineering Guide

How the copilot answers a question end to end, which models and data it uses, how the code is organised, and what the measurements say.

Build date: 2026-10-19

Corpus: 22 documents, 4,321 words

Benchmark: 168 questions (145 answerable)

Code: FastAPI + MySQL backend, Next.js 14 frontend, local Ollama model

Contents

Section	What it covers
1. What the product is	Problem, audience, the promise it keeps
2. End to end	One question, step by step through every layer
3. Surfaces	The eight screens and what each one is for
4. Roles and access	Capability matrix and where it is enforced
5. Models and algorithms	Every model, threshold and why it exists
6. Datasets	Corpus, synthetic records, benchmark, query log
7. Code structure	Module map for backend, frontend and scripts
8. API surface	Endpoints, request/response contracts, auth
9. Measurements	The evaluation report, read as a table
10. Limits and next steps	Known failure modes, stated plainly

1. What the product is

AMYPO answers questions about a campus from its own records and policy documents. The records are real rows in MySQL (students, attendance, grades, skills, companies and placement criteria); the policy text is a small markdown corpus written for this project. The product does not call any third-party API: retrieval, the language model, the verifier, the placement engine and the evaluator all run on the same machine, and the test suite fails the build if a socket opens to anything but loopback.

The four promises

- **Answers come from evidence.** Every claim is checked against the passages or rows it was built from, and citations name the record.
- **It abstains instead of inventing.** When retrieval cannot find support, the answer says so, names the closest records and asks a clarifying question.
- **Who asks decides what is visible.** The role comes from a signed session, never from the question text; students see their own record, staff see their department, admins see the system side.
- **It runs on modest hardware.** The default path is a 3.8B model on CPU, with a deterministic extractive fallback when no model is loaded, and the whole stack fits in an 8 GB budget.

2. End to end: one question, every layer

This is the path of a single `POST /api/v1/ask` call, in the order the code runs it. Each step records a stage in the trace, which the UI shows in the answer's trace panel.

Step	What happens
1. Session	The httpOnly access token is verified and turned into a Principal (user id, role, department). A forged or expired token is rejected before any question text is read.
2. Understanding	Rule-based intent classification (policy, personal record, aggregate, placement eligibility, skill gap, candidate recommendation, multi-hop) plus entity matching against real database vocab: course codes, student ids, company and role names. Every decision names the rule that fired.
3. Structured tools	For record intents, parameterised SQL templates run against MySQL. The student id is fixed by the session, so a student cannot widen the query, and rows come back with their record ids for citation.
4. Placement engine	Eligibility, skill-gap and what-if questions are decided by the rules engine, not generated: criteria from role_requirements are compared with the student's CGPA, backlogs, attendance and skill levels, and the answer cites the rows it used.
5. Retrieval	Hybrid search over 6,300-plus chunks: BM25 (rank_bm25) and a dense index (sentence-transformers when installed, a hash embedder otherwise), fused with reciprocal rank fusion at k=60, then a feature reranker. Access filtering happens before ranking.
6. Relevance gate	The best BM25 score retrieval produced is compared with the configured retrieval_relevance_floor (9.0). Below it, the pipeline abstains before generating: cheaper, and honest about the corpus not covering the question.
7. Generation	Either the local model (phi3:mini through Ollama) or, when no model is available, the deterministic extractive answer that quotes the retrieved passages in order. SQL rows can also be phrased without the model.
8. Verification	Every claim is split out and matched against the evidence. A number that is not in the evidence is a hard failure. If the supported share or confidence falls under the thresholds, the answer becomes an abstention.
9. Deliver	The response carries the answer, citations, confidence, route, cost estimate, carbon estimate, detected language, conflict notice, and a trace id. The run is written to the query log and to the semantic cache under a role- and user-scoped key.

3. The surfaces

The frontend is eight routes in Next.js 14 (app router). Navigation is built from the capability map, so a screen a role cannot use never appears; a direct URL shows a 'Not permitted' screen instead of an empty page.

Route	Purpose
/login	Sign in, honest error states, demo accounts panel
/	Chat: ask, citations, trace panel, history
/placement	Eligibility, skill gap, what-if, cohort view, PDF export
/dashboard	Live resources, route mix, latency, cost, quality, scenarios
/knowledge	Corpus status, ingestion trigger, coverage by scope
/evaluation	Run the harness, headline metrics, open the report
/audit	Cohort queries over student records
/admin	Users, roles, audit log, service keys

4. Roles and where access is enforced

Capability	Student	Staff	Admin
Public policy documents	yes	yes	yes
Staff-only documents	no	yes	yes

Capability	Student	Staff	Admin
Admin-only documents	no	no	yes
Own grades, attendance, CGPA	yes	no	aggregate
Department student records	no	assigned only	aggregate
Placement eligibility, skill gap, what-if	own	cohort	cohort
Edit company criteria	no	assigned only	yes
Users, ingestion, router, evaluation	no	no	yes
Audit log (admin view)	no	no	yes
Cohort-query trail	no	own queries	all

Enforcement is in three places and none of them is the model: route dependencies (`deps.require`) reject the request before the handler runs; the retriever filters chunks by document scope and by student id before ranking; SQL templates and the placement engine take the student id from the session, so the question text cannot substitute another student. Cache keys include the scope, so a cached staff answer can never be served to a student.

> Red-team result: 0 leaks across 161 cases (68 prompts and 93 requests, at least 30 per role). The suite asks a student for another student's grades and for staff-only policies, asks every role to ignore its permissions, and replays forged, expired and re-signed tokens. A leak is a 2xx where the matrix requires 401/403, or scoped data in the body of a 2xx.

5. Models and algorithms

There are three families of machine learning in the product, and each one is small enough to explain: a language model for phrasing, a classifier for routing, and lexical/vector retrieval for finding evidence. Everything else is deterministic rules, which is deliberate.

Model or method	Role	Details
phi3:mini (Ollama)	Phrase the grounded answer	3.8B parameters, Q4_0, about 2.2 GB resident; 2.5-3.1 s per answer on this CPU-only machine; temperature 0.1; the prompt pins it to the provided passages and forbids outside knowledge.
tinyllama	Tier 2 fallback	Used when RAM drops under 1400 MB and the smaller model is present; not pulled on the reference machine, so the tier is inert there.
Extractive fallback	Answer without a model	Quotes the top passages verbatim with citation markers. Deterministic, so every number in the evaluation is reproducible; it cannot paraphrase.
Complexity classifier	Route a question	scikit-learn pipeline over structural features (length, digits, cue counts), trained on 276 derived labels plus thumbs feedback; held-out accuracy 0.91, macro-F1 0.87, saved as a pickle and reloaded at start.
BM25	Lexical retrieval	rank_bm25 over chunk text, tokenised with stopwords removed and light stemming; strong on exact policy wording and record ids.
Dense retrieval	Semantic retrieval	384-dimensional sentence embeddings when sentence-transformers is installed; otherwise a hash embedder, which measures lexical overlap and is fused at weight 0 (measured: fusing it reduced success@1).
Reciprocal rank fusion	Merge the two rankings	score = sum of weight / (60 + rank); rank-based so the two score scales never need calibrating.
Feature reranker	Order the top candidates	Title match, section match, term coverage, recency and record type; MRR 0.789 against 0.780 with dense weighting.
Grounding verifier	Decide whether to trust the answer	Splits the answer into claims, matches each against the evidence (similarity floor 0.45), requires numbers to appear, and abstains when confidence is under 0.55.

Model or method	Role	Details
Semantic cache	Answer repeats cheaply	Keyed by normalised question, role and student scope; cosine floor 0.92; pre-warmed with 25 frequent questions per scope.

Routing policy

The router picks the cheapest route that can answer: cache (0.03-0.4 ms), extractive fallback (about 35 ms), local model (2.6 s, free), simulated cloud (1.8 s, 1.75 INR per call, 100 calls a day, 3% simulated failure). It refuses the cloud leg below 800 kbps bandwidth or when the quota is exhausted, drops to the smaller model under 1400 MB of free RAM, and goes fully offline under 900 MB. These constants are inputs to the evaluation, not hidden behaviour.

6. Datasets

All data is generated locally from a fixed seed (20260203) by `scripts/generate_data.py`, so a fresh clone can rebuild the corpus and the records exactly.

Dataset	Rows	What it holds
students	300	300 students: department, year, CGPA, backlogs
courses	17	17 courses with credits, semester and faculty
attendance	1250	1,250 student-course attendance records
grades	2500	2,500 assessment rows with weights
enrollments	1250	1,250 enrolments with grade and project grade
companies	15	15 recruiters with sector and tier
company_roles	20	20 roles with package and cut-offs
role_requirements	80	80 skill requirements per role
skills	34	34 skills with category
student_skills	3301	3,301 student-skill levels with evidence
schedules	17	17 weekly class slots

Documents and chunks

The corpus is 22 markdown documents (4,321 words): policy, exam rules, course manuals, a notice board, an FAQ, and three restricted documents (a faculty handbook and placement internals for staff, a security runbook for admins). Ingestion chunks the prose by heading (137 document chunks) and writes one compact record card per student and per attendance row (6,204 chunks), each carrying an access scope. Every chunk row keeps its text, token count, content hash and source id, so re-ingestion only re-embeds what actually changed.

> The record cards are what make a student's own attendance citable, but they are 98% of the chunks: terms like 'attendance' appear in thousands of cards, which flattens their BM25 weight for policy questions. Section 10 states the consequence and the fix.

Benchmark and query log

The evaluation benchmark holds 168 questions (145 answerable, 23 deliberately unanswerable): adversarial 10, factoid 20, multi_hop 15, numeric 60, placement 10, policy 35, unanswerable 18. Each item carries the expected answer, the gold source ids and its kind, which is what makes per-kind reporting possible. A separate log of 200 historical questions drives the routing corpus and the cache pre-warm, and the live route decision log accumulates every real request for the self-training loop.

7. Code structure

Backend: Python 3.12, FastAPI, Pydantic v2 and MySQL through a pooled connector. The container is built from `backend/Dockerfile` on `python:3.12-slim`; every module below is imported by the app and covered by the test suite.

Module	Responsibility
api/routers	FastAPI routes: ask, auth, placement, monitor, ingest, eval, admin
api/ask_service.py	The answering pipeline and the trace it produces
auth/	Passwords, JWT sessions, role dependencies, audit records
retrieval/	Intent rules, tokenizer, hybrid index (BM25 + vectors + rerank)
grounding/	Claim verifier, conflict detection, prompt-injection checks
placement/	Eligibility rules, skill gaps, suitability ranking, PDF export
router/	Complexity classifier, routing policy, metrics, self-training
llm/	Ollama client with tiers, prompts, SQL-tool answer templates
cache/	Semantic answer cache keyed by normalised question and scope
ingest/	Markdown and CSV ingestion, chunking, record cards
monitor/	RAM, CPU, bandwidth and simulated-cloud quota sampling
eval/	Benchmark, harness, ablations, sweeps, red-team suite, report
db/	MySQL connection, migrations runner, query helpers
core/	Settings, typed contracts, container wiring, logging

Each router is a thin HTTP layer over a service: validation and dependencies live in the route, decisions live in the module it calls. The answering pipeline (`api/ask_service.py`) is shared by `POST /ask`, the trace endpoint and the evaluation harness, so the numbers in the report come from the same code users hit.

Tests and scripts

Path	What it does
backend/tests/	Contract, retrieval, grounding, placement, router, auth/RBAC, evaluation, access-control suite; sockets to anything but loopback raise
scripts/generate_data.py	Builds the synthetic records, documents and benchmark from the seed
scripts/evaluate.py	The make-eval entry point: full harness run, writes report.json/html
scripts/make_artifacts.py	Architecture diagram, final summary PDF, Postman collection
scripts/make_guides.py	This document and the user manual
scripts/e2e_smoke.py	Live end-to-end check against a running stack
db/migrations/	Schema: init, cohort queries, auth (users, staff assignments, refresh tokens, audit)
docker-compose.yml	MySQL, backend, frontend, Ollama with memory limits and healthchecks

8. API surface

The OpenAPI document (`openapi.yaml`) is the contract: the typed response models in the backend are generated from the same shapes the frontend consumes. Every path below is served under `/api/v1`.

Endpoint	Purpose	Role
POST /ask	Ask a question; returns answer, citations, trace id	any signed-in
GET /ask/trace/{id}	Full reasoning trace of a previous request	any signed-in
POST /route	Classify a question and see the chosen route	any signed-in
GET /placement/eligibility	Eligibility for the caller or a cohort	any signed-in
GET /placement/skill-gap/{id}	Missing skills against a target role	self or staff

Endpoint	Purpose	Role
POST /placement/what-if	Score a hypothetical grade or skill change	self or staff
POST /placement/match	Rank candidates for a role	staff, admin
GET /placement/eligibility/{id}/report.pdf	Downloadable eligibility report	self or staff
GET /resource-status	RAM, CPU, bandwidth, quota, model state	open
GET /health	Component health and corpus size	open
GET /metrics	Aggregate quality and cost metrics	staff, admin
POST /ingest	Ingest the corpus, with hashes and record counts	admin
POST /eval/run	Run the evaluation harness	admin
POST /auth/login	Sign in; sets the httpOnly session cookies	open
POST /auth/refresh	Rotate the refresh token	signed-in session
GET /auth/me	Current user, role and department	any signed-in
GET/POST/PATCH /admin/users	Create, list and update accounts	admin
GET /admin/audit	Audit log of denials, logins and record access	admin

Authentication is local: bcrypt password hashes, a short-lived JWT access token and a revocable refresh token, both in httpOnly cookies, with a CSRF-safe same-site policy. Login is rate-limited and answers with a deliberately generic message; every denied action is written to the audit log with the actor, the route and the reason.

9. Measurements

These figures come from `docs/eval/report.json`, the artefact `make eval` writes. The run replays the benchmark through the real pipeline in the extractive configuration, so it is reproducible without a model; the report also carries the model-path numbers when a model is present.

Metric	Value	How it is measured
Answer accuracy	75.2%	109 correct of 145 answerable; gold citation or text match, with every expected number required
Groundedness	100.0%	Share of claims the verifier could support from the evidence
Hallucination rate	8.3%	14 of 168 delivered answers were unanswerable or ungrounded
Abstention (unanswerable)	39.1%	9 of 23 correctly abstained
Retrieval MRR	0.798	Hybrid plus reranker, first relevant passage
Success@1	75.9%	A gold passage ranked first
Routing agreement	100.0%	Chosen route against the oracle label
Cost saving	98.9%	3.36 INR against 294.00 INR if every question went to the cloud
Peak RSS	188 MB	psutil, sampled during the whole run
Red-team leaks	0	Must be zero; see section 4

The relevance floor, measured per point

Each row is a real answer pass at that floor, not an estimate. That is the trade the product ships: accuracy is bought against abstention, and the shipped value is marked.

Floor	Accuracy	Correct / answered	Hallucinated	Abstention
0.0	75.2%	109 / 142	10.1%	26.1%
4.5	75.2%	109 / 138	10.1%	26.1%
9.0 (shipped)	75.2%	109 / 135	8.3%	39.1%

Floor	Accuracy	Correct / answered	Hallucinated	Abstention
10.0	73.1%	106 / 132	6.6%	52.2%
12.0	73.1%	106 / 122	3.0%	78.3%
15.0	69.7%	101 / 113	2.4%	82.6%
18.0	68.3%	99 / 108	1.2%	91.3%

10. Limits, stated plainly

• **The model path is only as good as what is pulled.** With no model in the Ollama volume the pipeline answers extractively: honest and citable, but it cannot paraphrase and every number in the report comes from that path. In the containerised setup the pull must land in the Ollama *container's* volume; the `ollama-pull` service does that on `docker compose up`, and `make models` repeats it. A pull from a host-installed Ollama app goes to a different store and does not count.

• **Vague questions abstain.** 'What is the attendance policy?' is below the relevance floor; 'What is the minimum attendance required for exams?' answers. This is a corpus-scoring artefact, not a coincidence: record cards dominate the index and flatten BM25 weights for common policy words. The fix is a document-only score for the gate, or embeddings that separate meaning from wording.

• **List-shaped placement questions are weaker than specific ones.** 'Am I eligible for the Zoho SDE role?' is answered from the rules engine; 'Which companies am I eligible for?' can still abstain because the verifier wants the statement supported literally.

• **Dense retrieval is effectively off** without sentence-transformers, so the reported hybrid numbers equal BM25.

• **The cloud leg is simulated.** Quota, price, latency and failure rate are constants; no request leaves the machine.

• **Classifier labels are derived from a rule**, not human annotation, so its accuracy measures agreement with that rule. Only thumbs feedback adds non-derived signal.

• **Language support is experimental.** Script detection labels a question, but answers stay in English and no translated retrieval is attempted.